คู่มือ Deploy แอป Nuxt

สารบัญเอกสาร[1] · English[2] · Deploy Contract[3] · คู่มือผู้ดูแล[4]

ตรวจสอบจากซอร์สโค้ดในโปรเจกต์วันที่ **28 ตุลาคม 2026** (Asia/Bangkok) คู่มือนี้ครอบคลุมแอป Nuxt 4, API และ worker ตรวจสอบธุรกรรม ระบบซื้อเลเวลปัจจุบันใช้ **Polygon Amoy, chain ID 80002 และ tUSDT สำหรับทดสอบ** การเปิดเว็บผ่านโดเมนสาธารณะไม่ได้เปลี่ยน contract ให้เป็น mainnet

1. รูปแบบระบบและสิ่งที่ต้องเตรียม

```
Browser / connected wallet
          |
    HTTPS reverse proxy
          |
    Nuxt Node server ---- PostgreSQL
          |
    Polygon Amoy RPC ---- ReferralDistributor + AmoyTestUSDT
```

เตรียม Node.js **24+**, npm, PostgreSQL **17+**, ฐานข้อมูลที่สร้างไว้แล้ว, โดเมน และจุดรับ HTTPS ต้องใช้ Node process ที่ทำงานต่อเนื่อง เพราะระบบตรวจสอบธุรกรรมค้างทุก **15 วินาที** การใช้ static hosting จาก `npm run generate` ไม่สามารถรัน API และ worker เหล่านี้ได้ หากแพลตฟอร์มหยุด process ระหว่าง request ต้องออกแบบ worker แยกเพิ่มเติม ซึ่งยังไม่มีในโปรเจกต์นี้

รันคำสั่งจาก root ของโปรเจกต์ ตัวอย่างหลักใช้ Bash บน Linux หากใช้ Windows PowerShell ให้เปลี่ยน `npm` เป็น `npm.cmd`

เก็บ source และ lockfile ของ release ไว้ด้วย ไฟล์ `compose.yaml` ในโปรเจกต์มีเฉพาะ **PostgreSQL สำหรับ development** ยังไม่ใช่ชุด deploy แอป production ต้องจัดเตรียมรหัสฐานข้อมูลและระบบ backup สำหรับ production แยกต่างหาก

2. ติดตั้งและตั้งค่า

บนเครื่อง build หรือเครื่องผู้ deploy ให้ติดตั้ง dependencies รวม development tools ที่ใช้ build แอปและ compile contract:

```
npm ci --include=dev
test -f .env || cp .env.example .env
```

คำสั่ง PowerShell สำหรับติดตั้งและสร้างไฟล์ตั้งค่าโดยไม่เขียนทับไฟล์เดิม:

```
npm.cmd ci --include=dev
if (-not (Test-Path -LiteralPath .env)) { Copy-Item -LiteralPath .env.example -Destination .env }
```

แก้ `.env` เป็นการส่วนตัว ตัวอย่างด้านล่างเป็นแม่แบบ ต้องเปลี่ยนโดเมน ข้อมูลฐานข้อมูล และเติมค่าฝั่ง blockchain ก่อนเปิดให้ซื้อเลเวล:

```
NODE_ENV=production
NITRO_HOST=127.0.0.1
NITRO_PORT=3000
DATABASE_URL=postgresql://APP_USER:APP_PASSWORD@DB_HOST:5432/APP_DATABASE
NUXT_PUBLIC_APP_URL=https://app.example.com
NUXT_PUBLIC_WALLET_MODE=external
NUXT_PUBLIC_PASSWORD_LOGIN_ENABLED=false

NUXT_EVM_RPC_URL=https://rpc-amoy.polygon.technology/
NUXT_EVM_CHAIN_ID=80002
NUXT_EVM_TOKEN_ADDRESS=
NUXT_EVM_EXPLORER_API_KEY=
NUXT_REFERRAL_PAYMENT_MODE=contract
NUXT_REFERRAL_CONTRACT_ADDRESS=
NUXT_REFERRAL_AUTHORITY_PRIVATE_KEY=
NUXT_REFERRAL_MIN_CONFIRMATIONS=2
NUXT_REFERRAL_MIN_GAS_BALANCE=0.003
```

หากใช้ hosted PostgreSQL ให้ตั้ง SSL ตามผู้ให้บริการ เช่น `?sslmode=verify-full` เมื่อรองรับและตั้ง certificate ครบแล้ว อักขระพิเศษใน username/password ต้องทำ URL encoding ฐานข้อมูลต้องมีอยู่ก่อน และ role ของแอปต้องมีสิทธิ์ทำ migration รวมถึงอ่าน/เขียนตารางแอป จำกัดการเข้าถึงฐานข้อมูลให้อยู่ในเครือข่ายของแอปและผู้ดูแล

`NUXT_PUBLIC_APP_URL` ต้องตรงกับ origin ที่ browser เปิดจริง ทั้ง scheme, hostname และ port หากใช้ port พิเศษ ให้ redirect โดเมนอื่นมาที่โดเมนหลัก ตัวแปร `NUXT_PUBLIC_*` แสดงถึงฝั่ง browser ได้ จึงเก็บ key ไว้เฉพาะการตั้งค่าฝั่ง server

หากใช้ contract payments ให้ทำตาม คู่มือ Deploy Contract[3] เพื่อรับ token/distributor address และ authority key ที่ตรงกัน ระบบเดิมต้องคง address และ signer ของตัวเองไว้ แอปเริ่มทำงานได้ก่อนตั้งค่าเหล่านี้ครบ แต่ยังซื้อเลเวลไม่ได้

`NUXT_EVM_EXPLORER_API_KEY` ไม่จำเป็นต่อการเริ่มแอปหรือซื้อเลเวล แต่ประวัติ wallet ที่ดึงจาก explorer ต้องใช้ key ที่เข้าถึง Amoy ได้ ส่วน `NUXT_EVM_TREASURY_ADDRESS` และ `NUXT_EVM_MIN_CONFIRMATIONS` เป็นค่าของ generic transfer-evidence helpers แยกจากผู้รับค่าธรรมเนียมของ distributor ดู รายการ environment ทั้งหมด[4]

3. เตรียมฐานข้อมูลและผู้ดูแล

ตรวจสอบว่า `DATABASE_URL` ชี้ฐานข้อมูลที่ต้องการ แล้วรัน:

```
npm run db:status
npm run db:migrate
npm run admin:list
```

หากเป็นระบบใหม่และยังไม่มีผู้ดูแล ให้รัน:

```
npm run admin:create
```

คำสั่งจะถามรหัสผ่านความยาว **12–128 ตัวอักษร** และแสดง ID สมาชิกแบบตัวเลขที่ระบบสร้างให้ เก็บข้อมูลนี้อย่างปลอดภัยแล้วเข้าสู่ระบบที่ `/admin/login` ระบบไม่มีรหัสผู้ดูแลเริ่มต้น หากตั้งค่าแบบ noninteractive ใช้ `LOOP_ADMIN_PASSWORD` ได้ และควรถอดออกจาก runtime configuration หลังสร้างบัญชีเสร็จ

ผู้ดูแลยังเข้าด้วย ID สมาชิก/รหัสผ่านได้ แม้ปิด password login ของสมาชิกทั่วไป หากต้องการจัดการค่าธรรมเนียม contract ให้เข้าบัญชีผู้ดูแลก่อน แล้ว link external wallet ที่เป็นเจ้าของ distributor สิทธิ์ admin ของเว็บไม่ได้ให้สิทธิ์ owner ของ contract โดยอัตโนมัติ อ่านเรื่อง connected wallets[5] ก่อน link address ที่อาจถูกใช้กับบัญชีอื่นแล้ว

ตอนเริ่ม server ระบบจะทำ migration ที่ยังค้างจาก SQL ซึ่งรวมอยู่ใน `.output` ให้อีกครั้ง การรัน migration ก่อน rollout ช่วยให้ตรวจสอบข้อผิดพลาดได้ง่าย เก็บ SQL ที่ apply แล้วไว้ตามเดิม เพราะระบบตรวจชื่อไฟล์และ checksum การ reset ฐานข้อมูลไม่ใช่ขั้นตอน deploy

4. Build และเปิด Node server

Build release สำหรับ Node server โดยยังติดตั้ง development dependencies ไว้:

```
npm run typecheck
npm run build
node --env-file=.env .output/server/index.mjs
```

หากแพลตฟอร์ม build เลือก deployment preset อื่นให้อัตโนมัติ ให้กำหนด `NITRO_PRESET=node-server` ก่อน build บน Bash ใช้ `NITRO_PRESET=node-server npm run build` ส่วน PowerShell ใช้ `$env:NITRO_PRESET = 'node-server'` แล้วตามด้วย `npm.cmd run build`

ไฟล์ที่ใช้ deploy แอปคือ **โฟลเดอร์ `.output` ทั้งโฟลเดอร์** รวม `server`, `public` และ migrations ที่ bundle แล้ว ควร build บนแพลตฟอร์มที่เข้ากันกับเครื่องปลายทาง ส่วนคำสั่งสำหรับผู้ดูแล เช่น migration, สร้าง admin และ deploy contract ยังต้องใช้ source และ dependencies ของคำสั่งนั้น ไม่ได้เป็นคำสั่งภายใน `.output`

Production entry point ไม่อ่าน `.env` ให้อัตโนมัติ คำสั่งด้านบนจึงใช้ Node โหลดไฟล์อย่างชัดเจน หาก hosting inject environment ให้แล้ว ใช้ `node .output/server/index.mjs` และตั้ง `NODE_ENV=production` พร้อมตัวแปรที่จำเป็นผ่าน hosting ได้ ตัวแปรที่ process รับมาจากภายนอกมีลำดับความสำคัญเหนือ env file ของ Node จึงต้องเอาค่า override เก่าออกเมื่อเปลี่ยนการตั้งค่า อ้างอิง เอกสาร env file ของ Nuxt

Entry point และตัวแปร host/port ใช้ตาม เอกสาร Node runtime ของ Nitro หาก proxy อยู่เครื่องเดียวกันให้ bind `127.0.0.1` หาก container หรือ router ของ hosting ต้องเชื่อมเข้า process ให้ใช้ `0.0.0.0` พร้อมจำกัดการเข้าถึงในระดับเครือข่าย

5. รันต่อเนื่องบน Linux ด้วย systemd

ตัวอย่างนี้สมมติว่ามี service user ชื่อ **`unxo` อยู่แล้ว**, วาง release ที่ `/opt/unxo/app` และมี `.env` ส่วนตัวที่ user นี้อ่านได้ เปลี่ยน path และตำแหน่ง Node ให้ตรงกับเครื่องจริง ตรวจด้วย `command -v node` เพราะ service ไม่ได้โหลดการตั้งค่า Node version manager จาก interactive shell

สร้าง `/etc/systemd/system/unxo.service`:

```
[Unit]
Description=UNXO Nuxt server
Wants=network-online.target
After=network-online.target

[Service]
Type=simple
User=unxo
Group=unxo
WorkingDirectory=/opt/unxo/app
Environment=NODE_ENV=production
Environment=NITRO_HOST=127.0.0.1
Environment=NITRO_PORT=3000
ExecStart=/usr/bin/node --env-file=/opt/unxo/app/.env /opt/unxo/app/.output/server/index.mjs
Restart=always
RestartSec=5
NoNewPrivileges=true
UMask=0077

[Install]
WantedBy=multi-user.target
```

จำกัดสิทธิ์ `.env` ให้อ่านได้เฉพาะเจ้าของไฟล์ และตรวจว่า service user อ่านได้ จากนั้นเปิด service:

```
sudo systemctl daemon-reload
sudo systemctl enable --now unxo
sudo systemctl status unxo
sudo journalctl -u unxo -n 100 --no-pager
```

หาก deploy contract จากเครื่องอื่น ไม่ต้องเก็บ `REFERRAL_DEPLOYER_PRIVATE_KEY` ไว้บนเครื่องรันแอป แต่ runtime ยังต้องมี `NUXT_REFERRAL_AUTHORITY_PRIVATE_KEY` เดิมที่ตรงกับ contract เมื่อแก้ runtime settings ให้ restart service

6. โดเมน HTTPS และ reverse proxy

ตั้ง DNS ให้ชี้เซิร์ฟเวอร์และออก TLS certificate ที่ใช้ได้จริงผ่าน hosting หรือเครื่องมือจัดการ certificate ตัวอย่าง Nginx นี้สมมติว่ามีไฟล์ certificate แล้ว ต้องเปลี่ยนโดเมนและ path ก่อนเปิดใช้งาน:

```
server {
    listen 80;
    server_name app.example.com;
    return 301 https://app.example.com$request_uri;
}

server {
    listen 443 ssl;
    server_name app.example.com;
    ssl_certificate /etc/letsencrypt/live/app.example.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/app.example.com/privkey.pem;

    location / {
        proxy_pass http://127.0.0.1:3000;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-Host $host;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_cache off;
    }
}
```

เมื่อบันทึก site configuration ในตำแหน่งของ Nginx บนเครื่องแล้ว ให้ตรวจและ reload:

```
sudo nginx -t
sudo systemctl reload nginx
```

Proxy ต้องส่ง host สาธารณะและ HTTPS scheme ให้ถูกต้อง เพราะการตรวจ origin และ secure session cookie ใช้ข้อมูลเหล่านี้ Headers ใช้ตาม proxy directives ของ Nginx หาก CDN รับ TLS ก่อนส่งผ่าน proxy อีกชั้น ให้ตั้ง trusted proxy chain ให้คง scheme ต้นทางไว้ ไม่ควรเปิด backend port ให้เข้าจากภายนอกโดยตรง ปิด shared cache สำหรับ `/api/*` เพราะแอปส่ง `Cache-Control: no-store`

7. ตรวจสอบหลัง deploy

จากเครื่องภายนอก ตรวจโดเมนสาธารณะที่ใช้จริง:

```
curl -fsS https://app.example.com/ -o /dev/null
curl -i https://app.example.com/api/auth/session
curl -fsS https://app.example.com/api/platform-settings
```

บน Windows ใช้ `curl.exe` และเปลี่ยน `/dev/null` ในคำสั่งแรกเป็น `NUL` โปรเจกต์ไม่มี route `/api/health` โดยเฉพาะ หากยังไม่ login คำขอ session ควรได้ HTTP 200 พร้อม `{"user":null}` และ `Cache-Control: no-store` ส่วน platform settings ทดสอบการอ่านฐานข้อมูล การตรวจชุดนี้ยังไม่ได้ยืนยันว่า blockchain configuration พร้อมใช้งาน

- เปิดหน้าเว็บทั้งภาษาไทยและอังกฤษ แล้วเข้าผู้ดูแลที่ `/admin/login`
- เชื่อม wallet สมาชิกบน Amoy และเซ็น login ผ่านโดเมน HTTPS หลัก ตรวจว่า reload แล้วยังมี session และ cookie มี `Secure` กับ `HttpOnly`
- ทำ รายการตรวจ contract[3] ให้ครบ แล้วใช้สมาชิกทดสอบที่มี tUSDT และ test POL ซื้อเลเวล 1 รอ confirmation และตรวจประวัติซื้อ/บัญชี
- ตรวจว่า service restart ได้และ worker ยังตรวจสอบธุรกรรมค้างต่อได้ หากไม่มี explorer key ควรกระทบเฉพาะ indexed history ไม่ใช่การ login

8. อัปเดต กู้คืน และแก้ปัญหา

ก่อนอัปเดตให้บันทึก revision ของ release และ contract addresses สำรอง PostgreSQL, private runtime configuration และ `.data/contracts` บนเครื่องผู้ deploy จัดการธุรกรรมค้างให้เสร็จก่อนเปลี่ยน contract address หรือ payment mode

Build release ใหม่แยกจากโฟลเดอร์ที่กำลังรับ traffic ตรวจความเข้ากันได้ของ migration หยุดรับการเขียนข้อมูลในช่วงสลับเมื่อจำเป็น ทำ migration สลับไปใช้ artifact ใหม่ทั้งชุด แล้ว restart และตรวจตามหัวข้อก่อนหน้า หากใช้ systemd ตามตัวอย่าง ให้รัน `sudo systemctl restart unxo` หลังสลับ release

เก็บ artifact ก่อนหน้าไว้สำหรับ rollback แอป แต่ต้องตรวจว่ารองรับ schema หลัง migration แล้ว ระบบไม่มี down-migration อัตโนมัติ การ restore ฐานข้อมูลต้องเป็นขั้นตอนกู้คืนที่ตั้งใจและตรวจเทียบกิจกรรมบน chain ก่อน เพราะการ restore ไม่ได้ย้อน token transfers เก็บ authority key เดิมไว้ตลอดการ deploy แทนการสร้างใหม่ทุก release

อาการ	สิ่งที่ต้องตรวจ
Server เริ่มไม่ได้	เวอร์ชัน Node, path/สิทธิ์ของ service, การเชื่อมฐานข้อมูล และ migration errors ใน log
HTTP 502	Node service ทำงานหรือไม่, host/port ที่รับ request และ proxy upstream
Wallet login ได้ 403/503	`NUXT_PUBLIC_APP_URL` ต้องตรง, forwarded host/scheme และ environment เก่าใน process
Cookie หรือ write request ถูกปฏิเสธ	HTTPS, proxy headers, Origin ของ browser และโดเมนหลัก
Contract ใช้ไม่ได้	Amoy RPC/80002, token/distributor ที่ถูกต้อง, version 2 และ quote signer ที่ตรงกัน
การซื้อค้าง	เก็บ purchase ID/hash ตรวจ RPC/receipt/confirmations และรอ reconciliation ก่อนส่งเงินซ้ำ
Migration history/checksum ไม่ตรง	คืน SQL ที่ apply แล้วให้ตรงต้นฉบับ และเพิ่ม forward migration ใหม่เมื่อต้องเปลี่ยน schema

ซอร์สอ้างอิง: package scripts[6], runtime config[7], แม่แบบ environment[8], เริ่มฐานข้อมูล[9], reconciliation worker[10], authentication[11]

เอกสารและซอร์สอ้างอิงในโปรเจกต์

เลขอ้างอิงในเนื้อหาชี้ไปยังไฟล์ใน repository ตามรายการด้านล่าง ควรแนบ source เมื่อแชร์เอกสารที่ต้องใช้ไฟล์เหล่านี้ร่วมด้วย ลิงก์เว็บไซต์ภายนอกยังเปิดได้จาก PDF

1. `docs/README.md`
2. `docs/en/deploy-nuxt.md`
3. `docs/th/deploy-contracts.md`
4. `docs/th/operations.md`
5. `docs/connected-wallets.md`
6. `package.json`
7. `nuxt.config.ts`
8. `.env.example`
9. `server/plugins/database.ts`
10. `server/plugins/referral-reconciliation.ts`
11. `server/utils/auth.ts`